FSCT 8561 – LAB0

Reflection Question 1

1. What is a socket?

   - A socket is an endpoint for communication between two processes over a network or locally on the same host. It is identified by an IP address and a port number.

2. What does AF_INET mean?

   - AF_INET stands for address family internet and specifies the ipv4 protocol.

3. What does SOCK_STREAM mean?

   - SOCK_STREAM specifies a stream-oriented socket, which uses TCP.

Part 4 – Create a Basic TCP Server

```python
# Import the socket module
import socket

# Define server address and port
HOST = "127.0.0.1"
PORT = 12345

# Create a TCP socket (IPv4, TCP stream)
server_socket = socket.socket(
    socket.AF_INET,
    socket.SOCK_STREAM
)

# Bind the socket to the host address and port
server_socket.bind((HOST, PORT))
# Listen for incoming client connections (queue up to 1)
server_socket.listen(1)

print("Server is waiting for a connection...")

# Wait and accept an incoming client connection
client_socket, client_address = server_socket.accept()
print("Connected by:", client_address)

# Receive up to 1024 bytes of data from the client
data = client_socket.recv(1024)

# Decode received bytes into a string
message = data.decode()
print("Client says:", message)

# Prepare reply message
reply = "Message received"

# Send encoded reply back to the client
client_socket.send(reply.encode())

# Close client and server sockets
client_socket.close()
server_socket.close()

```

```
Problems  Output  Debug Console  Terminal  Ports

PS C:\Users\billy\Desktop> python .\server.socket.py
Server is waiting for a connection...
```

Part 5 – Create a Basic TCP Client

```python
# Import the socket module
import socket

# Define server address and port to connect to
HOST = "127.0.0.1"
PORT = 12345

# Create a TCP socket (IPv4, TCP stream)
client_socket = socket.socket(
    socket.AF_INET,
    socket.SOCK_STREAM
)

# Connect to the server
client_socket.connect((HOST, PORT))

# Prepare message to send
message = "Hello Server"

# Send encoded message to the server
client_socket.send(message.encode())

# Receive reply from the server that is up to 1024 bytes
response = client_socket.recv(1024)

# Decode bytes and print the server's reply
print("Server replied:", response.decode())

# Close the client socket connection
client_socket.close()

```

Problems Output Debug Console Terminal Ports

```
PS C:\Users\billy\Desktop> python .\server.socket.py
Server is waiting for a connection...
Connected by: ('127.0.0.1', 49312)
Client says: Hello Server
PS C:\Users\billy\Desktop>
```

Part 6:

Task A:

Client:

```python
# Import the socket module
import socket

# Define server address and port to connect to
HOST = "127.0.0.1"
PORT = 12345

# Create a TCP socket (IPv4, TCP stream)
client_socket = socket.socket(
    socket.AF_INET,
    socket.SOCK_STREAM
)

# Connect to the server
client_socket.connect((HOST, PORT))

# Prepare message to send
message = "Hello Server, this is Billy Z"

# Send encoded message to the server
client_socket.send(message.encode())

# Receive reply from the server that is up to 1024 bytes
response = client_socket.recv(1024)

# Decode bytes and print the server's reply
print("Server replied:", response.decode())

# Close the client socket connection
client_socket.close()

```

```
Problems  Output  Debug Console  Terminal  Ports

PS C:\Users\billy\Desktop> python .\server.socket.py
Server is waiting for a connection...
Connected by: ('127.0.0.1', 58463)
Client says: Hello Server, this is Billy Z
PS C:\Users\billy\Desktop>
```

Task B:

Server:

```python
# Import the socket module
import socket

# Define server address and port
HOST = "127.0.0.1"
PORT = 12345

# Create a TCP socket (IPv4, TCP stream)
server_socket = socket.socket(
    socket.AF_INET,
    socket.SOCK_STREAM
)

# Bind the socket to the host address and port
server_socket.bind((HOST, PORT))
# Listen for incoming client connections (queue up to 1)
server_socket.listen(1)

print("Server is waiting for a connection...")

# Wait and accept an incoming client connection
client_socket, client_address = server_socket.accept()
print("Connected by:", client_address)

# Receive up to 1024 bytes of data from the client
data = client_socket.recv(1024)

# Decode received bytes into a string
message = data.decode()
print("Client says:", message)

# Prepare reply message
reply = "Received your message Billy, thanks"

# Send encoded reply back to the client
client_socket.send(reply.encode())

# Close client and server sockets
client_socket.close()
server_socket.close()

```

```
Problems  Output  Debug Console  Terminal  Ports

PS C:\Users\billy\Desktop> python .\client.socket.py
Received your message Billy, thanks
PS C:\Users\billy\Desktop>
```

Task C:

```python
# Import the socket module
import socket

# Define server address and port
HOST = "127.0.0.1"
PORT = 12345

# Create a TCP socket (IPv4, TCP stream)
server_socket = socket.socket(
    socket.AF_INET,
    socket.SOCK_STREAM
)

# Bind the socket to the host address and port
server_socket.bind((HOST, PORT))
# Listen for incoming client connections (queue up to 1)
server_socket.listen(1)

print("Server is waiting for a connection...")

# Wait and accept an incoming client connection
client_socket, client_address = server_socket.accept()
print("Connected by:", client_address)

# Receive up to 1024 bytes of data from the client
data = client_socket.recv(1024)

# Decode received bytes into a string
message = data.decode()
print("Client says:", message)

# Prepare reply message
reply = "Server received: " + message

# Send encoded reply back to the client
client_socket.send(reply.encode())

# Close client and server sockets
client_socket.close()
server_socket.close()
```

```python
# Import the socket module
import socket

# Define server address and port to connect to
HOST = "127.0.0.1"
PORT = 12345

# Create a TCP socket (IPv4, TCP stream)
client_socket = socket.socket(
    socket.AF_INET,
    socket.SOCK_STREAM
)

# Connect to the server
client_socket.connect((HOST, PORT))

# Prepare message to send
message = "Hello Server, this is Billy Z"

# Send encoded message to the server
client_socket.send(message.encode())

# Receive reply from the server that is up to 1024 bytes
response = client_socket.recv(1024)

# Decode bytes and print the server's reply
print(response.decode())

# Close the client socket connection
client_socket.close()
```

```
Problems  Output  Debug Console  Terminal  Ports

PS C:\Users\billy\Desktop> python .\client.socket.py
Server received: Hello Server, this is Billy Z
PS C:\Users\billy\Desktop>
```

Part 7:

```python
import socket

client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client.connect(("example.com", 80))

request = (
    "GET / HTTP/1.1\r\n"
    "Host: example.com\r\n"
    "Connection: close\r\n"
    "\r\n"
)

client.send(request.encode())

response = b""
while True:
    data = client.recv(4096)
    if not data:
        break
    response += data

print(response.decode(errors="replace"))
client.close()
```

```
Problems  Output  Debug Console  Terminal  Ports

PS C:\Users\billy\Desktop> python .\public.web_server.py
HTTP/1.1 200 OK
Date: Fri, 11 Sep 2026 05:10:09 GMT
Content-Type: text/html
Transfer-Encoding: chunked
Connection: close
Server: cloudflare
Last-Modified: Thu, 10 Sep 2026 21:00:47 GMT
Allow: GET, HEAD
Accept-Ranges: bytes
Age: 14265
cf-cache-status: HIT
CF-RAY: a3942a126e0889d5-YVR

22f
<!doctype html><html lang="en"><head><title>Example Domain</title><link rel="icon" href="data:,"><meta name="viewpo
rt" content="width=device-width, initial-scale=1"><style>body{background:#eee;width:60vw;margin:15vh auto;font-fami
ly:system-ui,sans-serif}h1{font-size:1.5em}div{opacity:0.8}a:link,a:visited{color:#348}</style></head><body><div><h
1>Example Domain</h1><p>This domain is for use in documentation examples without needing permission. Avoid use in o
perations.</p><p><a href="https://iana.org/domains/example">Learn more</a></p></div></body></html>

0


PS C:\Users\billy\Desktop>
```

Reflection Question 2

1. Which host did the client connect to?
   - example.com
   - From client.connect(("example.com", 80)))
2. Which port was used?

80

3. Why is port 80 relevant?
   - Port 80 is the default for unencrypted HTTP traffic.
4. What did the server send back?
   - HTTP/1.1 200 OK indicates that the request succeeded.
   - (<!doctype html><html lang="en"><head><title>Example Domain<....) This is the HTML document for the Example Domain web page.
   - There are metadata headers including Content-Type: text/html, Server: cloudflare, date, caching info, and Connection: close.

Part 8: Challenge

Socket.IO server running:



Client connecting:

```python
import socketio
sio = socketio.Client()

@sio.event
def connect():
    print("connection established")
    # Prompt user to type a message, then send it to the server
    message = input("Enter message: ")
    sio.emit('message', message)
    # Disconnect automatically after sending
    sio.disconnect()

@sio.event
def disconnect():
    print("disconnected from server")

sio.connect('http://localhost:8080')
sio.wait()
```

```
Problems 1  Output  Debug Console  Terminal  Ports

PS C:\Users\billy\Desktop\FSCT8561-Labs-Billy-Zandan\Lab0\socketio> python .\socketio_client.py
websocket-client package not installed, only polling transport is available
connection established
Enter message:
```

Message received by the server:

```
PS C:\Users\billy\Desktop\FSCT8561-Labs-Billy-Zandan\Lab0\socketio> python .\socketio_server.py
======== Running on http://0.0.0.0:8080 ========
(Press CTRL+C to quit)
Socket ID: WCQSWD88FA7m2JLlAAAB
Data: This is Billy Zandan A01371854
```

Client disconnecting:

```
PS C:\Users\billy\Desktop\FSCT8561-Labs-Billy-Zandan\Lab0\socketio> python .\socketio_client.py
websocket-client package not installed, only polling transport is available
connection established
Enter message: This is Billy Zandan A01371854
disconnected from server
PS C:\Users\billy\Desktop\FSCT8561-Labs-Billy-Zandan\Lab0\socketio>
PS C:\Users\billy\Desktop\FSCT8561-Labs-Billy-Zandan\Lab0\socketio>
```

FSCT 8561 – LAB0

1. What could happen if a server accepts connections from anyone?

- The server is binded to 0.0.0.0:8080, so it's listening on every network interface and will accept connections from any IP. That makes a few problems: anyone on the same network or if its exposed to the internet can connect and start interacting with it. An attacker could also just flood it with connections and knock it offline. And because there's no filtering, a malicious client could potentially trick the server into leaking data.

2. Is the Lab 0 message encrypted?

- No, the raw socket version just uses plain TCP on port 9090, and the Socket.IO version runs over plain HTTP on 8080. The message "This is Billy Zandan A01371854" travels in cleartext, so anyone who can see the traffic can read it.

3. Could network traffic be observed by another system?

- Yes, because everything is sent in plaintext, it's vulnerable to eavesdropping, man-in-the-middle attacks, and spoofing.

4. Why should servers validate input?

- Without validation, a huge payload can crash the server or exhaust the memory. Validation can block injection attacks, command injection, SQL injection, or worse depending on what the server does with the data. It can also check if incoming messages are the right type, length, and format that keeps the application from getting hacked.

5. Why might authentication be useful?

- Authentication would let the server confirm who's talking to it before doing anything, stop random or malicious clients from sending messages, make it possible to log who sent what, and allows it to block a single compromised user without taking the whole server down. Without some form of access control there is no way to tell a trusted user from an attacker.

6. Suggest two ways to make this application more secure.

- First, switch to TLS so the traffic is encrypted. Use HTTPS instead of plain HTTP by adding an SSL certificate to the aiohttp server. That stops eavesdropping and MITM attacks.

- Second, require an authentication token on connect and reject anything that doesn't supply a valid one. At the same time, validate message length and type before processing them.

FSCT 8561 – LAB0

Short Paragraph explaining what you learned and what was unclear:

What was unclear was the overall purpose of the lab since most of the code was already provided and ready to paste, a bit vague, it was hard to understand what I was supposed to be learning rather than just copying and running it. As someone who only knows a bit of Python, and especially for students in the class from the CISA program with little to no Python experience, the lab felt too long and jumped into complex concepts like async servers and Socket.IO without enough buildup. It would have been more helpful to have more hints or guidance that would let us students figure out the code themselves like step-by-step instructions to understand the code better, rather than having the full solution handed over from the start.

https://github.com/wotprizzleR6/FSCT8561-Labs-Billy-Zandan